

School of Advanced Technology

Final Year Project

Project Specification Report

Project Title	Quality-Aware Vector Retrieval: Detecting and Mitigating Embedding Corruption in Vector Databases
Student Name	Gaoxu.Li
Student ID	2362421
Project Field	Vector Databases / Data Quality / Information Retrieval
Supervisor	Dechang Xu
Co-supervisor (if applicable)	

Project Description and Problem Statement

Vector databases have become fundamental infrastructure for modern AI systems, powering retrieval-augmented generation (RAG), semantic search, and recommendation engines in virtually every large-scale deployment. However, a critical operational gap exists: when embeddings stored in these databases become silently corrupted through pipeline bugs, hardware errors, or configuration mistakes there is no built-in mechanism to detect or mitigate the damage. The VLDB Journal 2024 survey by Pan, Wang, and Li [1] explicitly identifies "vector quality and data drift" as an open problem, yet no systematic framework exists to study or address it.

This project investigates five corruption types that arise from distinct real-world failure modes:

- Additive noise:** Floating-point quantization errors, network transmission bit flips
- Dimension masking:** Truncated serialisation, field dropout in multi-model data pipelines
- Scale shift:** Missing L2-normalisation, unit inconsistency across embedding models
- Semantic displacement:** ID-to-vector mapping errors, stale caches, database race conditions
- Outlier injection:** Corrupted storage blocks, adversarial data poisoning

Recent work on adversarial attacks against vector databases including SilentRetrieval (KDD 2026) [2], the Black-Hole Attack [3], and VectorSmuggle [4] demonstrates that embedding manipulation can catastrophically degrade retrieval quality. The VectorSmuggle study [4] independently validates three of the five corruption types studied here (noise, scaling, offset perturbations) and confirms that distribution-shifting corruptions are detectable by Isolation Forest and One-Class SVM with AUROC at least 0.85, while geometry-preserving

manipulations evade statistical detection. The VDBMS Software Testing Roadmap [5] provides a complementary perspective from the software engineering angle, cataloguing implementation-level bugs but noting the absence of data-level quality assurance mechanisms.

This project fills this gap by constructing a unified benchmark framework (`corruption_bench`) to answer three interconnected research questions:

Q1. Retrieval Impact — How does each corruption type affect recall (Recall@k) under different index configurations (Flat exact search, HNSW approximate search)? Are some corruption types amplified by HNSW graph topology relative to exact search?

Q2. Detection Efficacy — Which detection methods (PCA reconstruction error, kNN-based local density, rule-based heuristics) best identify corrupted vectors? Where does each method fail, and why?

Q3. Detection Threshold and Early Warning — What is the minimum detectable corruption rate? Can detectors provide statistically significant warning before retrieval quality degrades below an operational threshold?

Aims and Objectives

The project has four interconnected aims, each with a concrete, verifiable success criterion:

Aim 1: Embedding Corruption Taxonomy. Establish the first systematic, code-based taxonomy of embedding data corruption in vector databases. Map each of the five corruption types to a documented real-world failure mode, define parameterisable severity levels, and provide a reproducible implementation via the `corruption_bench` framework. Success criterion: a literature-validated taxonomy where each corruption type has an independent reference supporting its real-world relevance.

Aim 2: Retrieval Impact Quantification. Measure Recall@k degradation across the full experimental grid: 5 corruption types x 6 corruption rates (0% to 50%) x 3 embedding dimensionalities (128, 256, 768) x 2 index configurations (Flat exact baseline, HNSW approximate search). Determine which corruption types are most damaging, whether higher dimensionalities amplify degradation, and whether HNSW graph topology mitigates or amplifies corruption effects relative to exact search. Success criterion: statistically significant Recall@k degradation curves for each experimental condition with confidence intervals from multiple random seeds.

Aim 3: Detection Method Benchmark and Improvement. Benchmark three unsupervised detection approaches (PCA-based Feature Reconstruction Error, kNN-based local density with reverse-kNN count, rule-based norm/NaN/zero-dimension checks) against all five corruption types using AUROC and precision-at-5%-FPR metrics. Analyse the theoretical reasons why all

unsupervised methods fail on semantic displacement a geometrically normal but semantically wrong corruption type. Address this gap by designing a label-propagation detector that leverages a small fraction (1 to 5%) of verified labels ("spot-checked" vectors, as done in operational quality audits) to identify vectors whose propagated label conflicts with their claimed label. Success criterion: label-propagation detector achieves AUROC above 0.7 on semantic displacement while requiring at most 5% seed labels.

Aim 4: Quality-Aware Retrieval Pipeline. Design and implement a closed-loop system where detection scores feed into a quality-aware re-ranking mechanism: HNSW retrieves top-2k candidates, detection-based quality scores apply a distance penalty, and the re-ranked top-k is returned to the user. Quantify Recall@k recovery (percentage of lost recall reclaimed) under varying penalty strength (lambda ablation study from lambda=0, no penalty, to lambda large, effectively filtered). Compare Flat (where pure soft-filtering modifies all vector distances before sorting) versus HNSW (where only post-retrieval re-ranking is feasible) to isolate index-structure limitations from data-quality effects. Success criterion: Recall@k recovery above 60% of the clean-baseline-to-corrupted gap for at least 4 of 5 corruption types.

Project Plan: Tasks and Milestones

The project spans 8 weeks (7 July - 31 August 2026), organised into four phases: infrastructure, detector design, experiments, and writing. Each phase has a concrete, verifiable milestone.

Week	Dates	Phase	Tasks	Milestone / Deliverable
W1	Jul 7-13	Infrastructure	Read core papers (HNSW [9], VLDB Survey [1], VDBMS Roadmap [5], Black-Hole [3], LOF [10]); implement FlatIndex exact-search baseline; download SIFT-128 (siftsmall) and GloVe-100 subsets	v2 tag: FlatIndex + real data loaders operational
W2	Jul 14-20	Detector	Read detection-method papers (PCA [11], SubspaceAD, NNAE+KNRN); implement label-propagation detector for semantic displacement; initial validation on synthetic Gaussian data	v3 tag: 4th detector implemented; target AUROC > 0.65 on semantic displacement
W3	Jul 21-27	Infrastructure	Implement quality-aware re-ranking module; integrate all detectors into unified pipeline; run Q1 (retrieval impact) on Synthetic + SIFT + GloVe across all	All infrastructure complete; Q1 results collected

W4	Jul 28 - Aug 3	Experiments	parameter grids Run Q2 (detector comparison, AUROC) on all 3 datasets x 5 corruption types; run Q3 (threshold analysis) with 25 fine-grained corruption rates for additive noise and semantic displacement	Q1, Q2, Q3 experimental data complete
W5	Aug 4-10	Experiments + Writing	Run Q4 (quality-aware retrieval, lambda ablation); run Q5 (label seed sensitivity, 1% to 10% seeds); generate all figures (Recall curves, ROC curves, recovery plots); write Introduction and Method sections	All experiments complete; paper skeleton drafted
W6	Aug 11-17	Writing	Complete Method (taxonomy, detectors, quality-aware pipeline) and Experiments sections; finalise all figures and tables; write Discussion (theoretical analysis of semantic displacement undetectability, practical implications)	Paper first draft complete
W7	Aug 18-24	Writing	Revise paper based on self-review: strengthen Related Work positioning, refine Limitations section, add MedMNIST + DINOv2 domain validation experiment if time permits	Paper second draft; submit to supervisor for feedback
W8	Aug 25-31	Finalisation	Address supervisor feedback; prepare project demonstration (live pipeline walkthrough); format manuscript per target venue guidelines; archive code and data for reproducibility	Final paper; project demonstration ready

Project Deliverables

1. corruption_bench Open-Source Benchmark Framework. A complete, well-documented Python package implementing the full experimental pipeline: data generation and loading (synthetic Gaussian, SIFT-128, GloVe-100), corruption injection (5 types with configurable

severity parameters), index construction (HNSW wrapper, Flat exact-search baseline), four detection methods (FRE, kNN, rule, label-propagation), evaluation metrics (Recall@k, AUROC, ROC curves), quality-aware re-ranking module, and visualisation. All parameters are centralised in a single configuration dataclass for full reproducibility. Standardised interfaces enable future extension (additional corruption types, detectors, or index backends such as FAISS IVF).

2. Comprehensive Experimental Results. Systematic evaluation across 5 corruption types, 6 rates, 3 dimensionalities, 2 index types, and 3 datasets (estimated 500+ experimental conditions). Core results include: (a) Recall@k degradation curves showing how each corruption type differentially affects retrieval quality, with HNSW vs. Flat comparison revealing index-structure amplification effects; (b) detector AUROC comparison matrices identifying which methods work for which corruption types; (c) detection threshold curves quantifying the "early warning window" between first detectable signal and significant retrieval degradation; (d) quality-aware retrieval recovery rates under varying lambda penalty strengths, with ablation showing the relative contribution of each detector to recovery.

3. Research Paper Manuscript. A complete research paper targeting a top-tier database or data engineering venue (VLDB / SIGMOD / ICDE). The paper contributes: (a) the first systematic taxonomy of embedding corruption in vector databases; (b) a theoretical analysis explaining why semantic displacement is undetectable by any purely unsupervised methods a negative result with practical implications; (c) a practical semi-supervised detection approach using label propagation that bridges this gap; and (d) a quality-aware retrieval pipeline that closes the loop from detection to retrieval improvement, validated on three datasets.

4. Project Demonstration. An end-to-end live walkthrough of the system: (1) generate synthetic data with visualised Gaussian clusters in 2D PCA projection; (2) inject corruption and highlight affected vectors; (3) demonstrate detector scoring distributions (clean vs. corrupted histograms) with AUROC metrics; (4) show quality-aware re-ranking recovering Recall@k in real time; (5) display key plots: Recall degradation curves across corruption rates, ROC curves for all detector-corruption type pairs, and detection lead-time heatmaps.

Project Industrial Relevance

Vector databases underpin the retrieval architecture of virtually every production RAG system, LLM-powered application, and semantic search engine deployed today (Pinecone, Weaviate, Qdrant, Milvus, Chroma, pgvector). When embeddings stored in these databases become corrupted a silent failure mode with no existing alert mechanism the downstream impact propagates through the entire application stack. A corrupted vector returned as a top-10 result in a RAG pipeline directly contaminates the LLM context with incorrect information [4].

This project directly addresses three industrial pain points:

No quality visibility. As reported by Help Net Security (May 2026) [4], most enterprises have "no visibility into the contents of the vector embeddings leaving their network, and no one is responsible for monitoring that channel." The detection methods benchmarked in this project provide the first systematic evaluation of practical quality monitoring approaches for stored embeddings.

No graceful degradation. Current vector databases either return corrupted vectors or delete them; there is no middle ground. The quality-aware re-ranking mechanism proposed here enables soft degradation: suspicious vectors are penalised but not removed, providing a tunable trade-off between recall and risk. In production settings where deletion is unacceptable (audit requirements, uncertain detection scores), soft filtering is the only viable option.

Index-agnostic design. The framework is designed to work with any distance-based index. The Flat exact-search baseline isolates pure data-quality effects from index-structure effects, while HNSW experiments validate the approach on the most widely deployed production index. This separation ensures that findings are not tied to a specific vector database implementation and can guide quality monitoring strategy regardless of the underlying index.

The practical outcome is a set of actionable recommendations for vector database operators: which detectors to deploy for which threat models, at what corruption rates detection becomes reliable, and how to integrate quality scores into retrieval without modifying index internals.

References

- [1] J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *The VLDB Journal*, vol. 33, no. 5, pp. 1591-1615, 2024.
- [2] Anonymous, "SilentRetrieval: Hijacking retrieval-augmented generation via semantically-preserving adversarial data poisoning," in *Proc. 30th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2026.
- [3] Anonymous, "Can you trust the vectors in your vector database? Black-hole attack from embedding space defects," *arXiv:2604.05480*, Apr. 2026.
- [4] J. Wanger, "VectorSmuggle: Steganographic exfiltration in embedding stores and a cryptographic provenance defense," *arXiv:2605.13764*, May 2026.
- [5] S. Wang et al., "Towards reliable vector database management systems: A software testing roadmap for 2030," *arXiv:2502.20812*, Feb. 2025.
- [6] M. Aumuller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Information Systems*, vol. 87, 2020.
- [7] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532-1543.
- [8] J. Yang et al., "MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, vol. 10, 2023.
- [9] Y. Malkov and D. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824-836, 2020.
- [10] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 93-104.

- [11] M.-L. Shyu, S.-C. Chen, K. Sarinnapakorn, and L. Chang, "A novel anomaly detection scheme based on principal component classifier," Dept. Elect. Comput. Eng., Univ. of Miami, Coral Gables, FL, Tech. Rep., 2003.
- [12] Y. Du, "Semantic certainty assessment in vector retrieval systems: A novel framework for embedding quality evaluation," arXiv:2507.05933, Jul. 2025.
- [13] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," Trans. Mach. Learn. Res., 2024.
- [14] Anonymous, "Reveal hidden pitfalls and navigate next generation of vector similarity search from task-centric views," in Proc. ACM SIGMOD Int. Conf. Management of Data (SIGMOD), 2026.